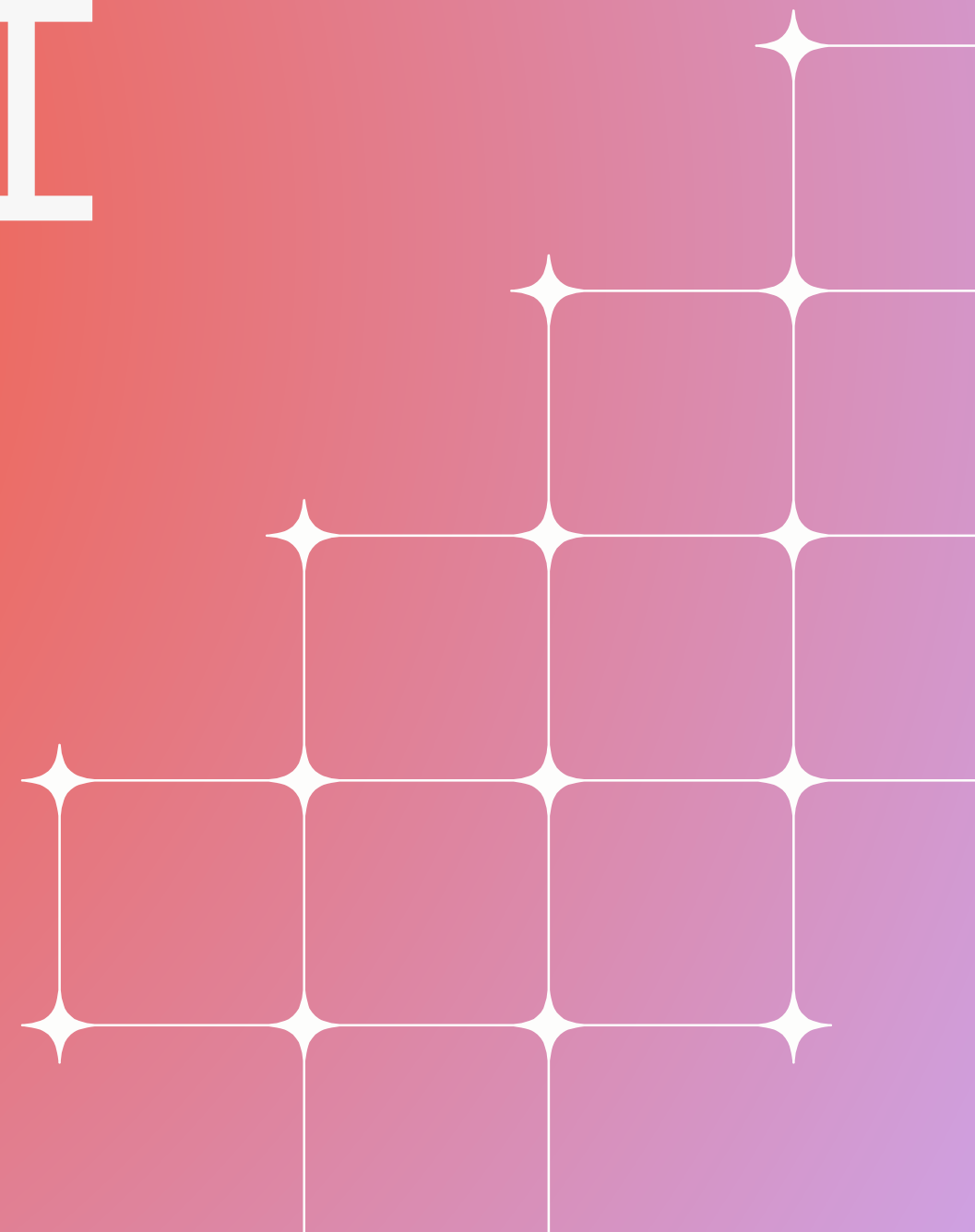


SERVERLESS + AI

Building Scalable Systems
Without Losing Your Mind



Joanne Skiles

AI IS EASY TO DEMO.

HARD TO PRODUCTIONIZE.

- LATENCY UNPREDICTABLE
- COSTS SPIKE
- INCONSISTENT OUTPUTS

WHAT YOU'LL WALK AWAY WITH

01

Fundamentals

02

Architecture
Patterns

03

What breaks

04

Design for scale

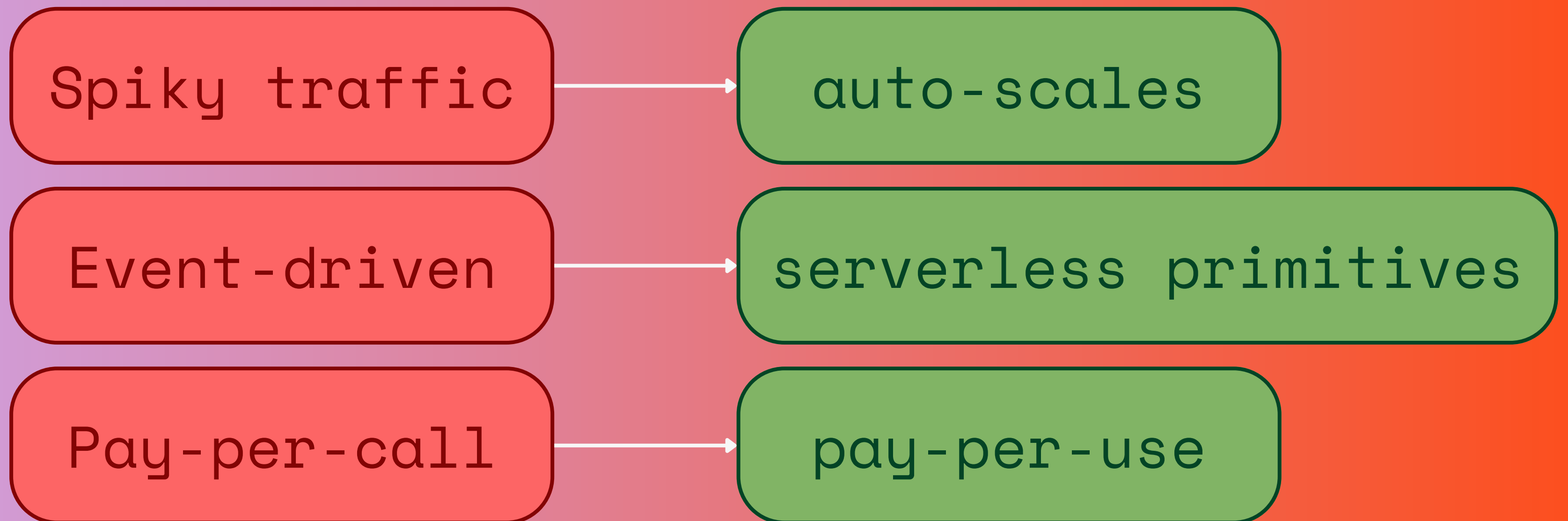
AI SYSTEMS **BREAK** ALL YOUR ASSUMPTIONS

Non-
deterministic

Variable
latency

External
dependencies

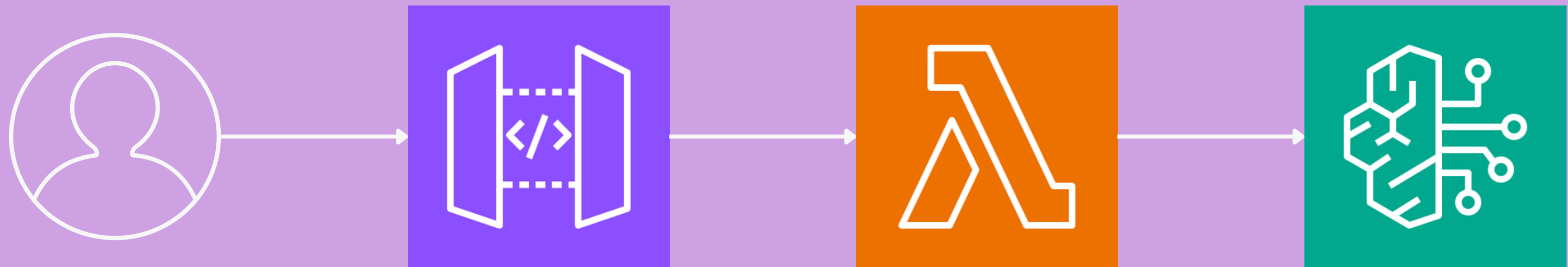
SERVERLESS EMBRACES UNPREDICTABILITY



YOUR CORE TOOLKIT

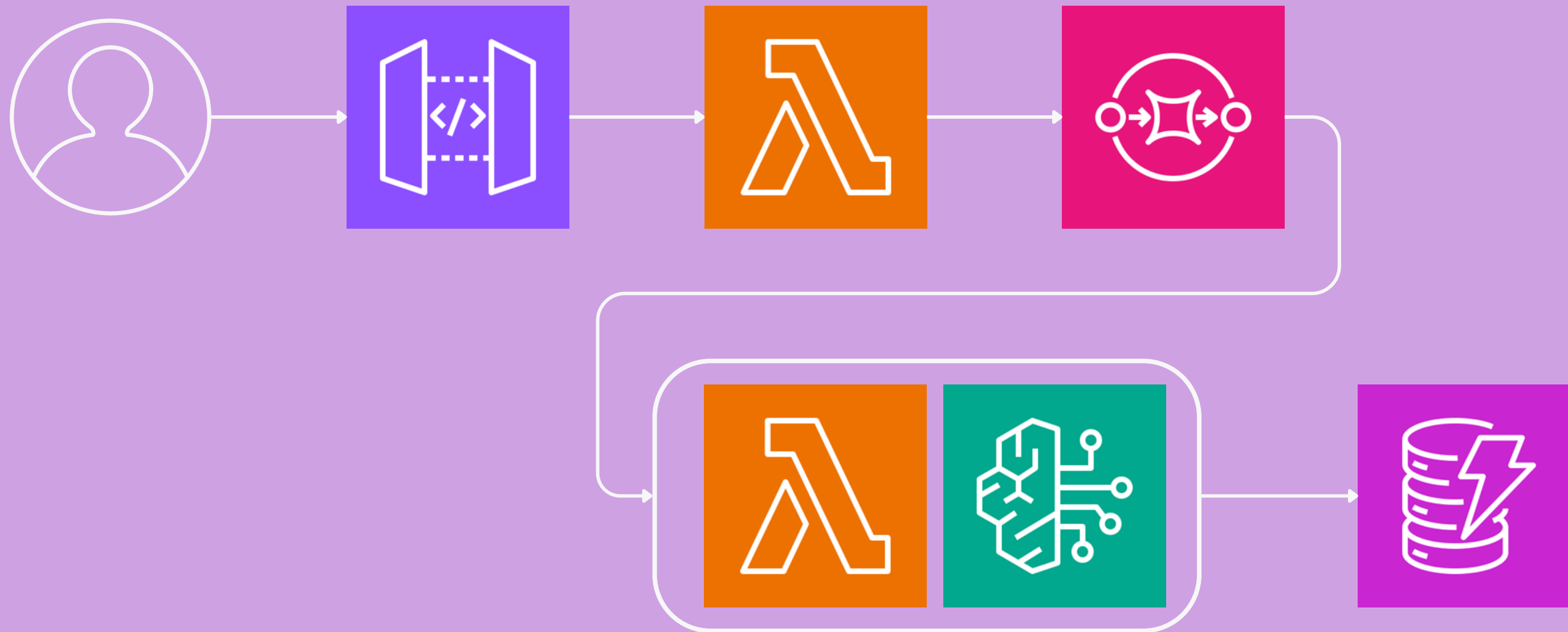
- LAMBDA — COMPUTE
- API GATEWAY — FRONT DOOR
- BEDROCK — MANAGED AI MODELS
- SQS — ASYNC QUEUE
- STEP FUNCTIONS — ORCHESTRATION

NAIVE ARCHITECTURE (SYNCHRONOUS)



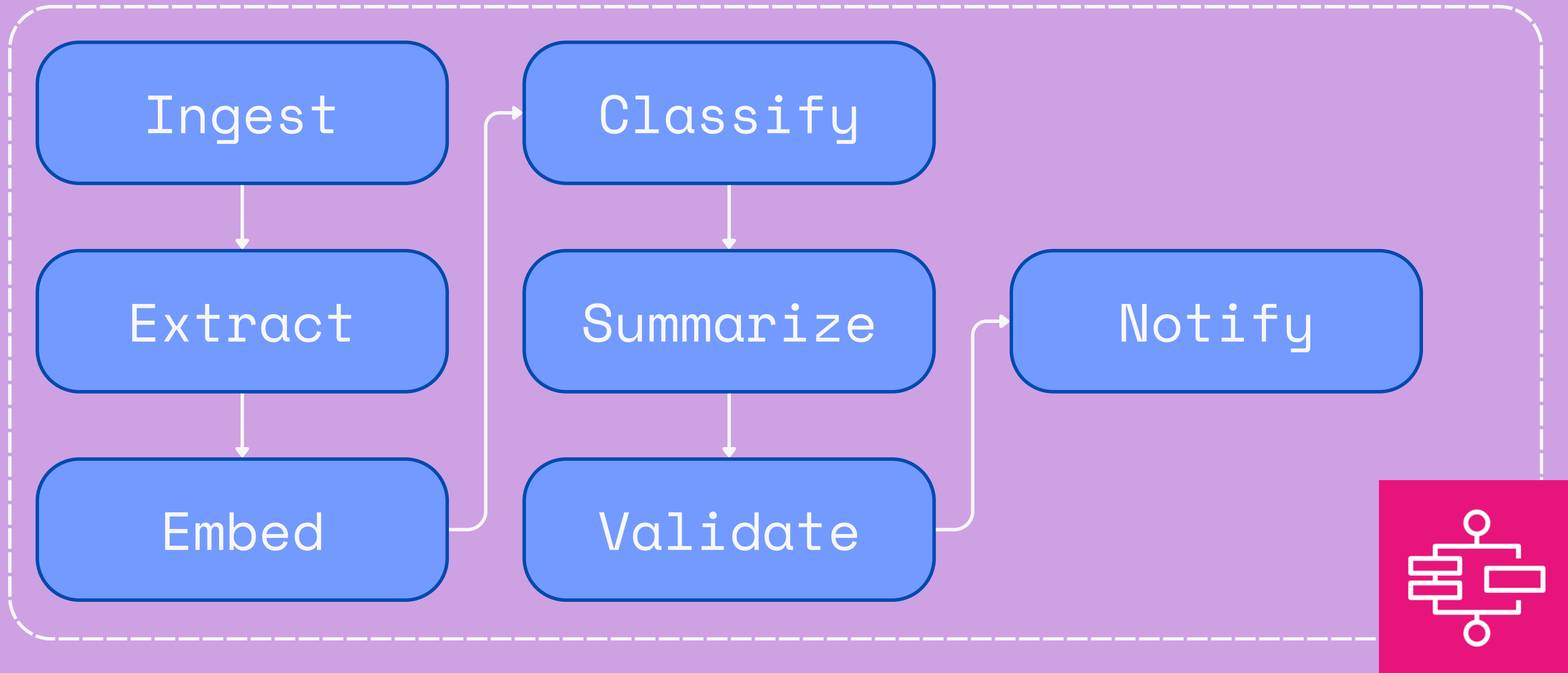
User waits 3–10s for response

SCALABLE ASYNC ARCHITECTURE



instant job ID → result polls later

ORCHESTRATED WORKFLOWS



DEMO

WHAT BREAKS IN PRODUCTION

Cost

Lantency

Output quality

Debugging

DESIGN FOR FAILURE EARLY

Caching

Async

Guardrails

Observability

You're not calling an
API.

You're designing a
distributed system.

5 RULES

1. MAKE IT ASYNC FIRST
2. LOG PROMPTS AND RESPONSES FROM DAY ONE
3. SET TOKEN BUDGETS DELIBERATELY
4. VALIDATE OUTPUTS BEFORE USERS SEE THEM
5. DESIGN FOR THE RETRY

SERVERLESS MAKES AI *SURVIVABLE* AT SCALE

JOANNE SKILES / DEMO REPO

<https://github.com/chaotictoejam/demo-serverless-ai-on-aws>

